STEP 1-EXPLORATORY DATA ANALYSIS

In [ ]:
```
# 1. IMPORT LIBRARIES
# Basic libraries
import pandas as pd
import numpy as np

# Visualizations
import matplotlib.pyplot as plt
import seaborn as sns

# Regression models
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor,
from sklearn.neighbors import KNeighborsRegressor
from sklearn.svm import SVR

# Evaluation
from sklearn.metrics import r2_score
```

In [ ]:
```
# 2. SET DISPLAY OPTIONS
# To show all columns in output
pd.set_option('display.max_columns', None)
```

In [ ]:
```
# 3. LOAD THE DATASET
df= pd.read_csv("Life Expectancy Data.csv")
```

In [ ]:
```
df
```

Out[ ]:

	Country	Year	Status	Life expectancy	Adult Mortality	infant deaths	Alcohol	per expe
0	Afghanistan	2015	Developing	65.0	263.0	62	0.01	7
1	Afghanistan	2014	Developing	59.9	271.0	64	0.01	7
2	Afghanistan	2013	Developing	59.9	268.0	66	0.01	7
3	Afghanistan	2012	Developing	59.5	272.0	69	0.01	7
4	Afghanistan	2011	Developing	59.2	275.0	71	0.01	
...	...	...	...	...	...	...	...	
2933	Zimbabwe	2004	Developing	44.3	723.0	27	4.36	
2934	Zimbabwe	2003	Developing	44.5	715.0	26	4.06	
2935	Zimbabwe	2002	Developing	44.8	73.0	25	4.43	
2936	Zimbabwe	2001	Developing	45.3	686.0	25	1.72	
2937	Zimbabwe	2000	Developing	46.0	665.0	24	1.68	

2938 rows × 22 columns

In [ ]:
```
# 4. BASIC CHECKS: HEAD, SHAPE, INFO, DESCRIBE, COLUMNS
df.head()             # First 5 rows
```

Out[ ]:

	Country	Year	Status	Life expectancy	Adult Mortality	infant deaths	Alcohol	percen expendi
0	Afghanistan	2015	Developing	65.0	263.0	62	0.01	71.27
1	Afghanistan	2014	Developing	59.9	271.0	64	0.01	73.52
2	Afghanistan	2013	Developing	59.9	268.0	66	0.01	73.21
3	Afghanistan	2012	Developing	59.5	272.0	69	0.01	78.18
4	Afghanistan	2011	Developing	59.2	275.0	71	0.01	7.09

In [ ]:
```
df.shape              # Rows & columns
```

Out[ ]:
```
(2938, 22)
```

In [ ]:
```
df.info()             # Dtypes + missing values
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2938 entries, 0 to 2937
Data columns (total 22 columns):
 #   Column                           Non-Null Count  Dtype  
---  ------                           --------------  -----  
 0   Country                          2938 non-null   object 
 1   Year                             2938 non-null   int64  
 2   Status                           2938 non-null   object 
 3   Life expectancy                  2928 non-null   float64
 4   Adult Mortality                  2928 non-null   float64
 5   infant deaths                    2938 non-null   int64  
 6   Alcohol                          2744 non-null   float64
 7   percentage expenditure           2938 non-null   float64
 8   Hepatitis B                      2385 non-null   float64
 9   Measles                          2938 non-null   int64  
 10   BMI                             2904 non-null   float64
 11  under-five deaths                2938 non-null   int64  
 12  Polio                            2919 non-null   float64
 13  Total expenditure                2712 non-null   float64
 14  Diphtheria                       2919 non-null   float64
 15   HIV/AIDS                        2938 non-null   float64
 16  GDP                              2490 non-null   float64
 17  Population                       2286 non-null   float64
 18   thinness  1-19 years            2904 non-null   float64
 19   thinness 5-9 years              2904 non-null   float64
 20  Income composition of resources  2771 non-null   float64
 21  Schooling                        2775 non-null   float64
dtypes: float64(16), int64(4), object(2)
memory usage: 505.1+ KB
```

In [ ]:
```
df.describe()        # Numeric summary
```

Out[ ]:

	Year	Life expectancy	Adult Mortality	infant deaths	Alcohol	perce expen
count	2938.000000	2928.000000	2928.000000	2938.000000	2744.000000	2938.0
mean	2007.518720	69.224932	164.796448	30.303948	4.602861	738.2
std	4.613841	9.523867	124.292079	117.926501	4.052413	1987.9
min	2000.000000	36.300000	1.000000	0.000000	0.010000	0.0
25%	2004.000000	63.100000	74.000000	0.000000	0.877500	4.6
50%	2008.000000	72.100000	144.000000	3.000000	3.755000	64.9
75%	2012.000000	75.700000	228.000000	22.000000	7.702500	441.5
max	2015.000000	89.000000	723.000000	1800.000000	17.870000	19479.9

In [ ]:
```
df.columns           # Column names
```

Out[ ]:
```
Index(['Country', 'Year', 'Status', 'Life expectancy ', 'Adult Mortality',
       'infant deaths', 'Alcohol', 'percentage expenditure', 'Hepatitis B',
       'Measles ', ' BMI ', 'under-five deaths ', 'Polio', 'Total expenditur
e',
       'Diphtheria ', ' HIV/AIDS', 'GDP', 'Population',
       ' thinness  1-19 years', ' thinness 5-9 years',
       'Income composition of resources', 'Schooling'],
      dtype='object')
```

In [ ]:
```
# 5. RENAMING ALL THE COLUMNS NAMES
df.rename(columns={
    'Country': 'Country',
    'Year': 'Year',
    'Status': 'Status',
    'Life expectancy ': 'Life_expectancy',
    'Adult Mortality': 'Adult_Mortality',
    'infant deaths': 'Infant_deaths',
    'Alcohol': 'Alcohol',
    'percentage expenditure': 'Percentage_expenditure',
    'Hepatitis B': 'Hepatitis_B',
    'Measles ': 'Measles',
    ' BMI ': 'BMI',
    'under-five deaths ': 'Under_five_deaths',
    'Polio': 'Polio',
    'Total expenditure': 'Total_expenditure',
    'Diphtheria ': 'Diphtheria',
    ' HIV/AIDS': 'HIV_AIDS',
    'GDP': 'GDP',
    'Population': 'Population',
    ' thinness  1-19 years': 'Thinness_1_19_years',
    ' thinness 5-9 years': 'Thinness_5_9_years',
    'Income composition of resources': 'Income_composition_of_resources',
    'Schooling': 'Schooling'
}, inplace=True)
```

In [ ]:
```
# RECHECK
df.columns
```

Out[ ]:
```
Index(['Country', 'Year', 'Status', 'Life_expectancy', 'Adult_Mortality',
       'Infant_deaths', 'Alcohol', 'Percentage_expenditure', 'Hepatitis_B',
       'Measles', 'BMI', 'Under_five_deaths', 'Polio', 'Total_expenditure',
       'Diphtheria', 'HIV_AIDS', 'GDP', 'Population', 'Thinness_1_19_years',
       'Thinness_5_9_years', 'Income_composition_of_resources', 'Schooling'],
      dtype='object')
```

In [ ]:
```
df.head()
```

Out[ ]:

	Country	Year	Status	Life_expectancy	Adult_Mortality	Infant_deaths
0	Afghanistan	2015	Developing	65.0	263.0	62
1	Afghanistan	2014	Developing	59.9	271.0	64
2	Afghanistan	2013	Developing	59.9	268.0	66
3	Afghanistan	2012	Developing	59.5	272.0	69
4	Afghanistan	2011	Developing	59.2	275.0	71

In [ ]:

```
# 6. MISSING VALUE CHECK
df.isnull().sum()      # Count missing
```

Out[ ]:

	0
Country	0
Year	0
Status	0
Life_expectancy	10
Adult_Mortality	10
Infant_deaths	0
Alcohol	194
Percentage_expenditure	0
Hepatitis_B	553
Measles	0
BMI	34
Under_five_deaths	0
Polio	19
Total_expenditure	226
Diphtheria	19
HIV_AIDS	0
GDP	448
Population	652
Thinness_1_19_years	34
Thinness_5_9_years	34
Income_composition_of_resources	167
Schooling	163

dtype: int64

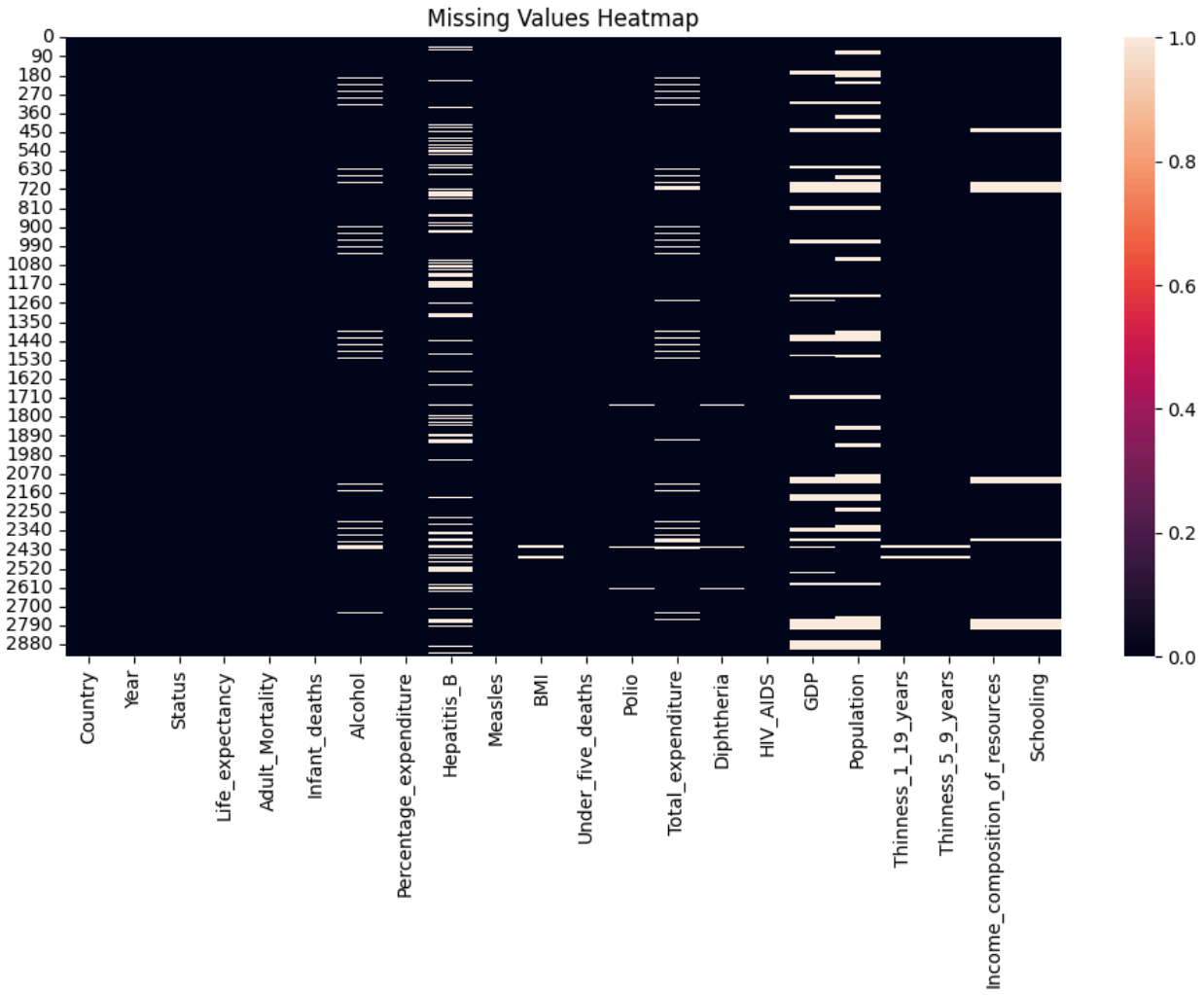
In [ ]:
```
df.notnull().sum()      # Count non-missing
```

Out[ ]:

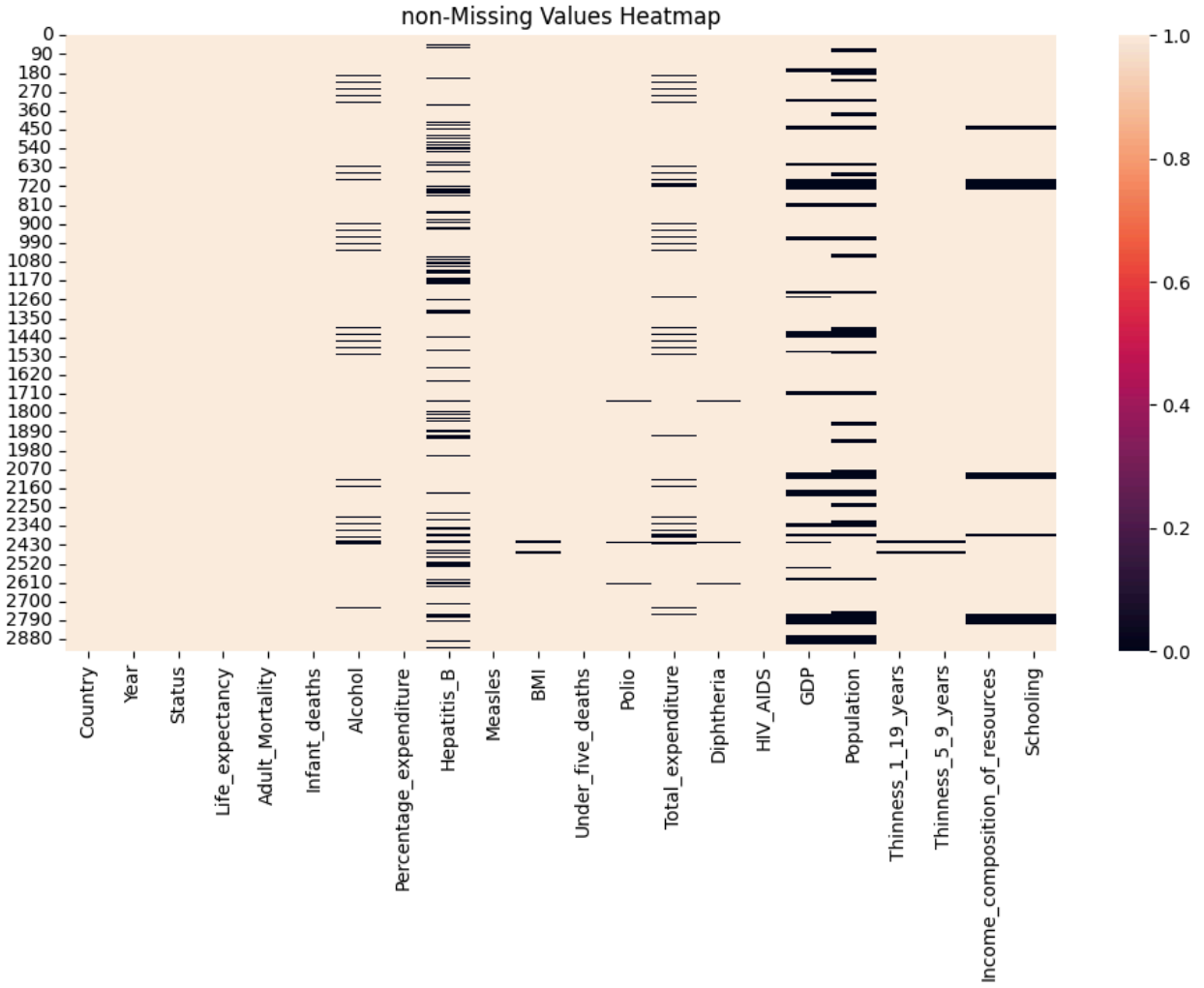
	0
Country	2938
Year	2938
Status	2938
Life_expectancy	2928
Adult_Mortality	2928
Infant_deaths	2938
Alcohol	2744
Percentage_expenditure	2938
Hepatitis_B	2385
Measles	2938
BMI	2904
Under_five_deaths	2938
Polio	2919
Total_expenditure	2712
Diphtheria	2919
HIV_AIDS	2938
GDP	2490
Population	2286
Thinness_1_19_years	2904
Thinness_5_9_years	2904
Income_composition_of_resources	2771
Schooling	2775

dtype: int64

In [ ]:
```
# 7. HEATMAP OF MISSING & NON-MISSING
plt.figure(figsize=(12,6))
sns.heatmap(df.isnull(), cbar=True)
plt.title("Missing Values Heatmap")
plt.show()
```



In [ ]:
```
plt.figure(figsize=(12,6))
sns.heatmap(df.notnull(), cbar=True)
plt.title("non-Missing Values Heatmap")
plt.show()
```



In [ ]:
```
# 8. REMOVE DUPLICATES
df.duplicated().sum()        # Show duplicates
```

Out[ ]:
```
np.int64(0)
```

In [ ]:
```
df = df.drop_duplicates()    # Remove duplicates
```

In [ ]:
```
df.duplicated().sum()        # Recheck
```

Out[ ]:
```
np.int64(0)
```

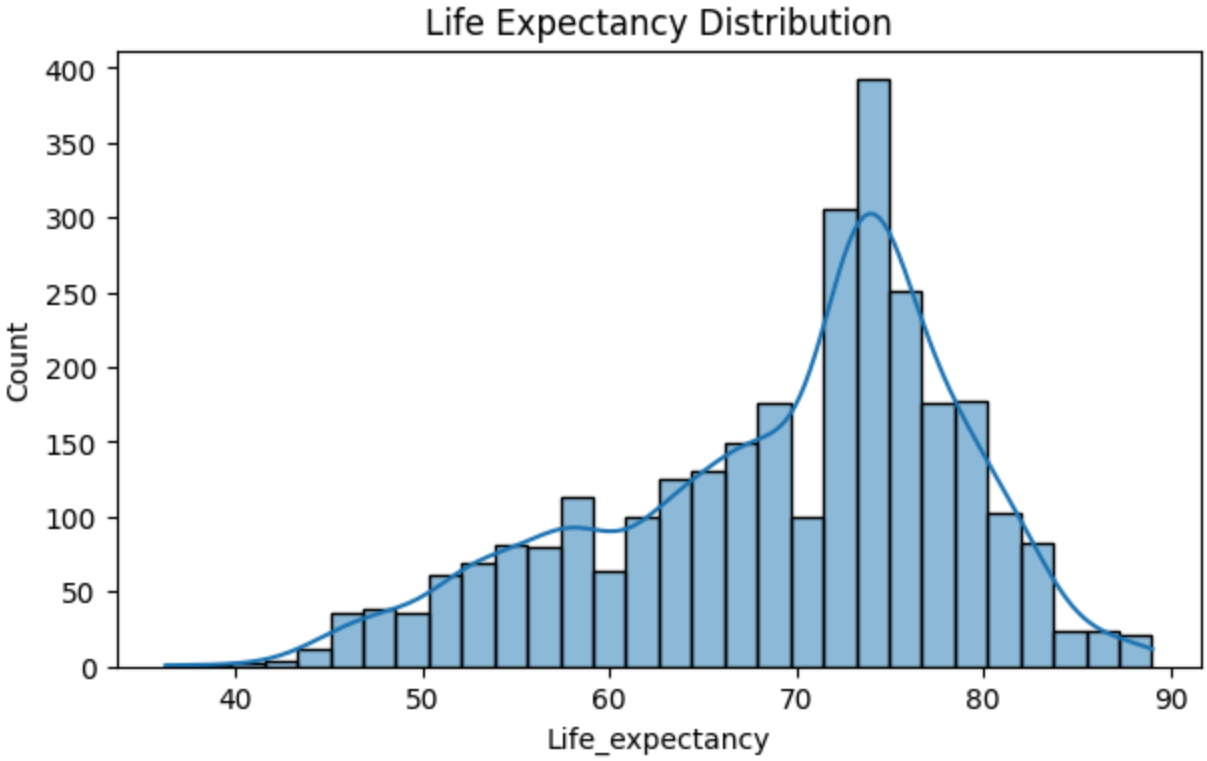
In [ ]:
```
# 9. DATATYPES CHECK
df.dtypes
```

Out[ ]:

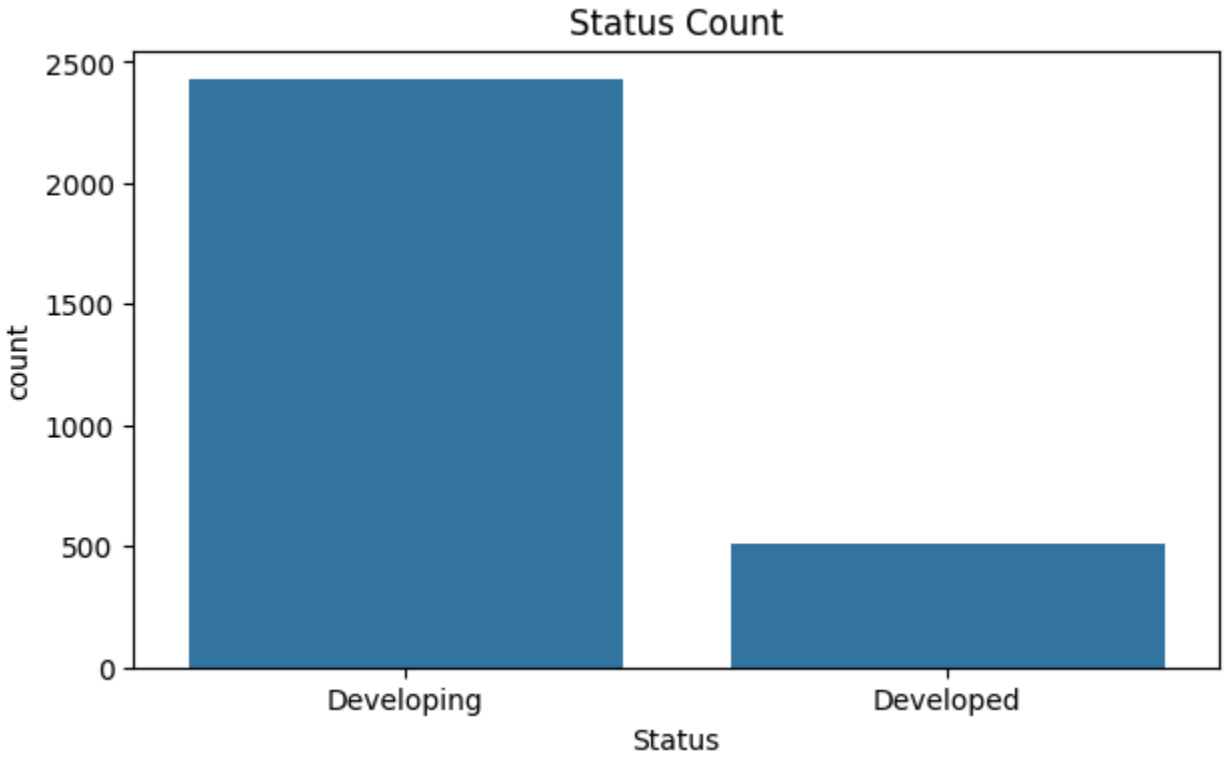
	0
Country	object
Year	int64
Status	object
Life_expectancy	float64
Adult_Mortality	float64
Infant_deaths	int64
Alcohol	float64
Percentage_expenditure	float64
Hepatitis_B	float64
Measles	int64
BMI	float64
Under_five_deaths	int64
Polio	float64
Total_expenditure	float64
Diphtheria	float64
HIV_AIDS	float64
GDP	float64
Population	float64
Thinness_1_19_years	float64
Thinness_5_9_years	float64
Income_composition_of_resources	float64
Schooling	float64

dtype: object

In [ ]:
```
# 10. UNI-VARIATE VISUALIZATIONS
# 1. Histogram
plt.figure(figsize=(7,4))
sns.histplot(df['Life_expectancy'], kde=True)
plt.title("Life Expectancy Distribution")
plt.show()
```



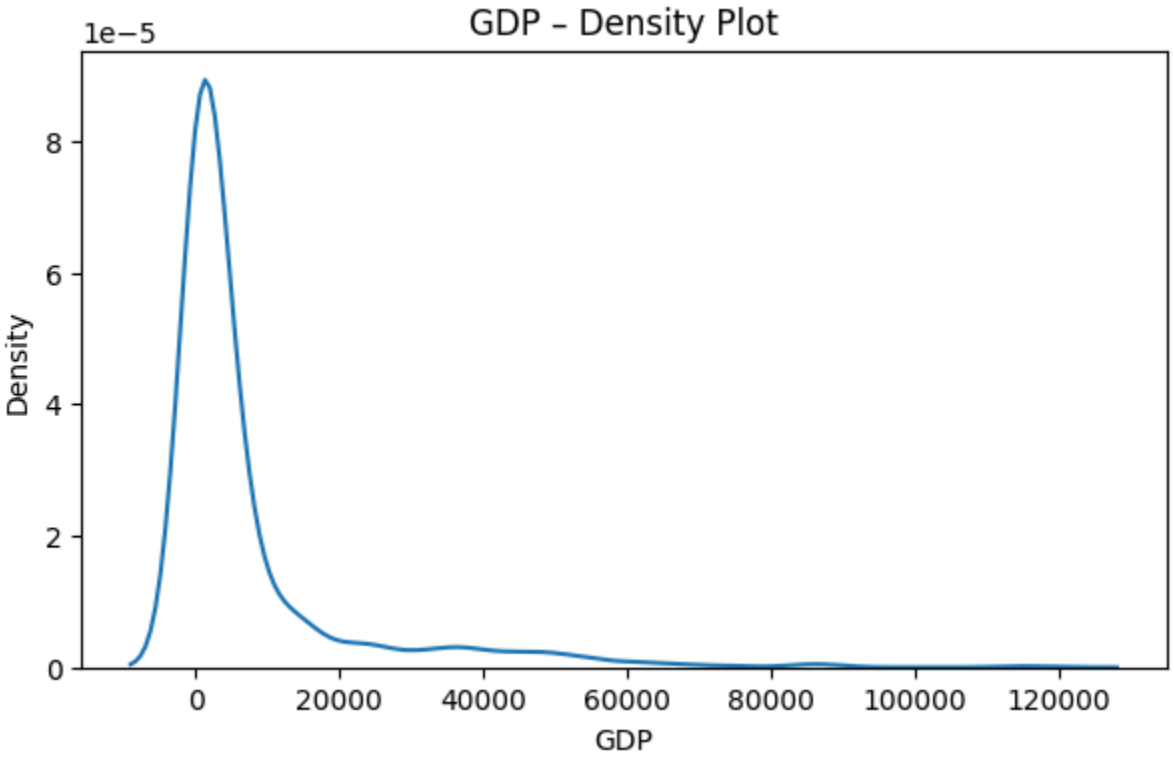
In [ ]:
```
# 2. Countplot
plt.figure(figsize=(7,4))
sns.countplot(x=df['Status'])
plt.title("Status Count")
plt.show()
```



In [ ]:
```
# 3. Boxplot
plt.figure(figsize=(7,4))
sns.boxplot(x=df['Life_expectancy'])
plt.title("Boxplot of Life Expectancy")
plt.show()
```

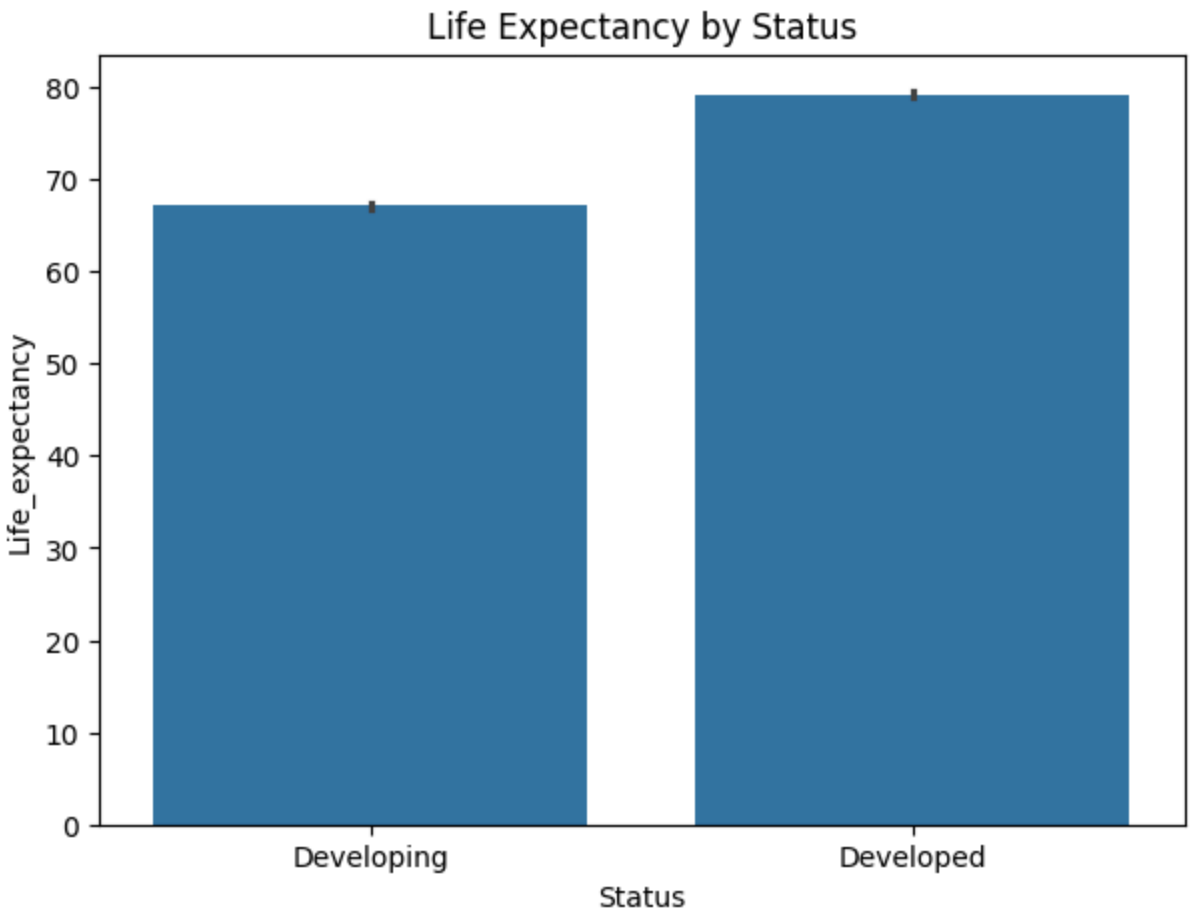
In [ ]:
```
# 4. Violin plot
plt.figure(figsize=(7,4))
sns.violinplot(x=df['Adult_Mortality'])
plt.title("Adult_Mortality – Violin Plot")
plt.show()
```

In [ ]:
```
# 5. KDE plot
plt.figure(figsize=(7,4))
sns.kdeplot(df['GDP'])
plt.title("GDP – Density Plot")
plt.show()
```

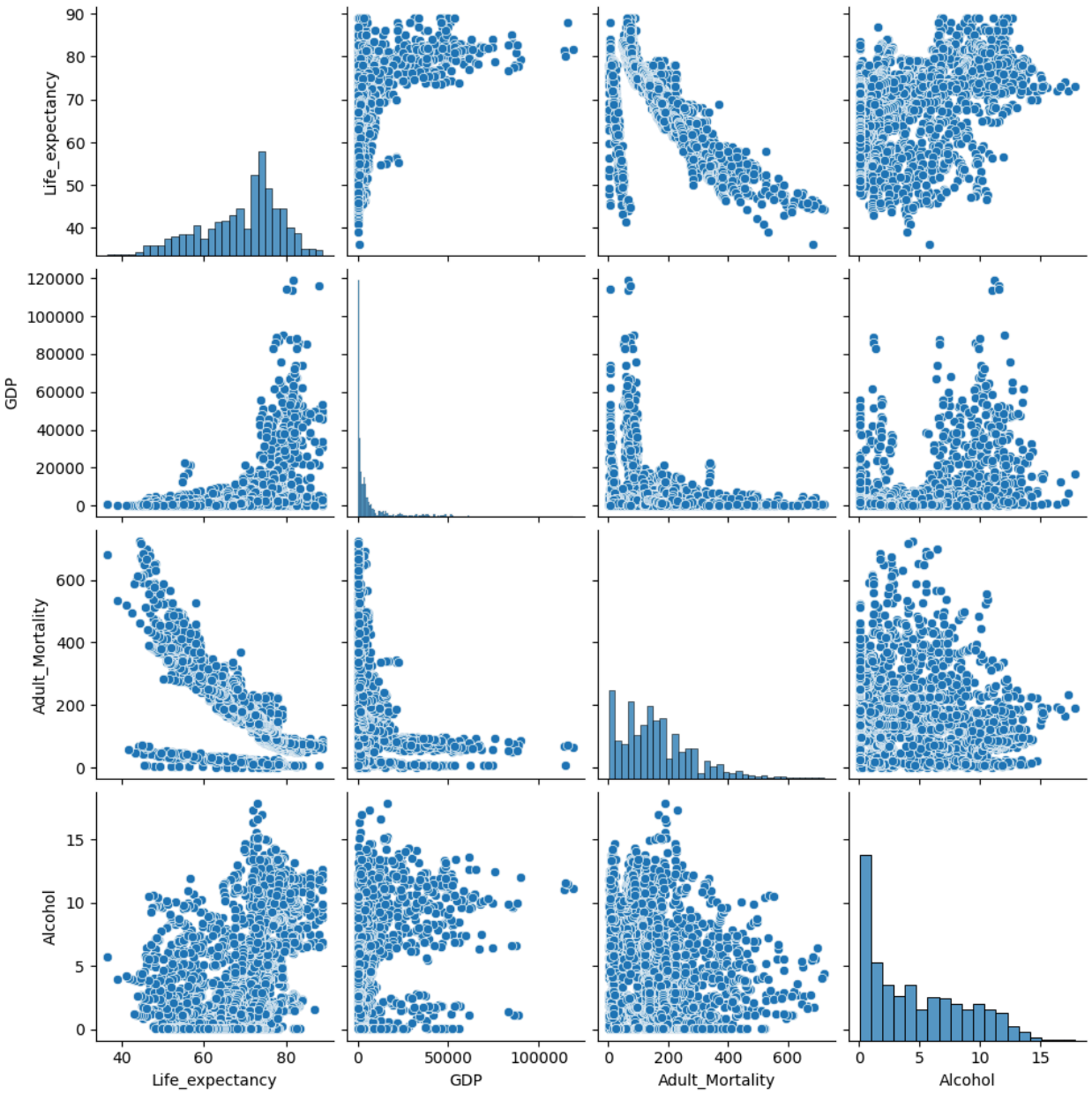


In [ ]:
```
# 11. BI-VARIATE VISUALIZATIONS
# 6. Scatterplot
plt.figure(figsize=(7,5))
sns.scatterplot(x=df['GDP'], y=df['Life_expectancy'])
plt.title("GDP vs Life Expectancy")
plt.show()
```

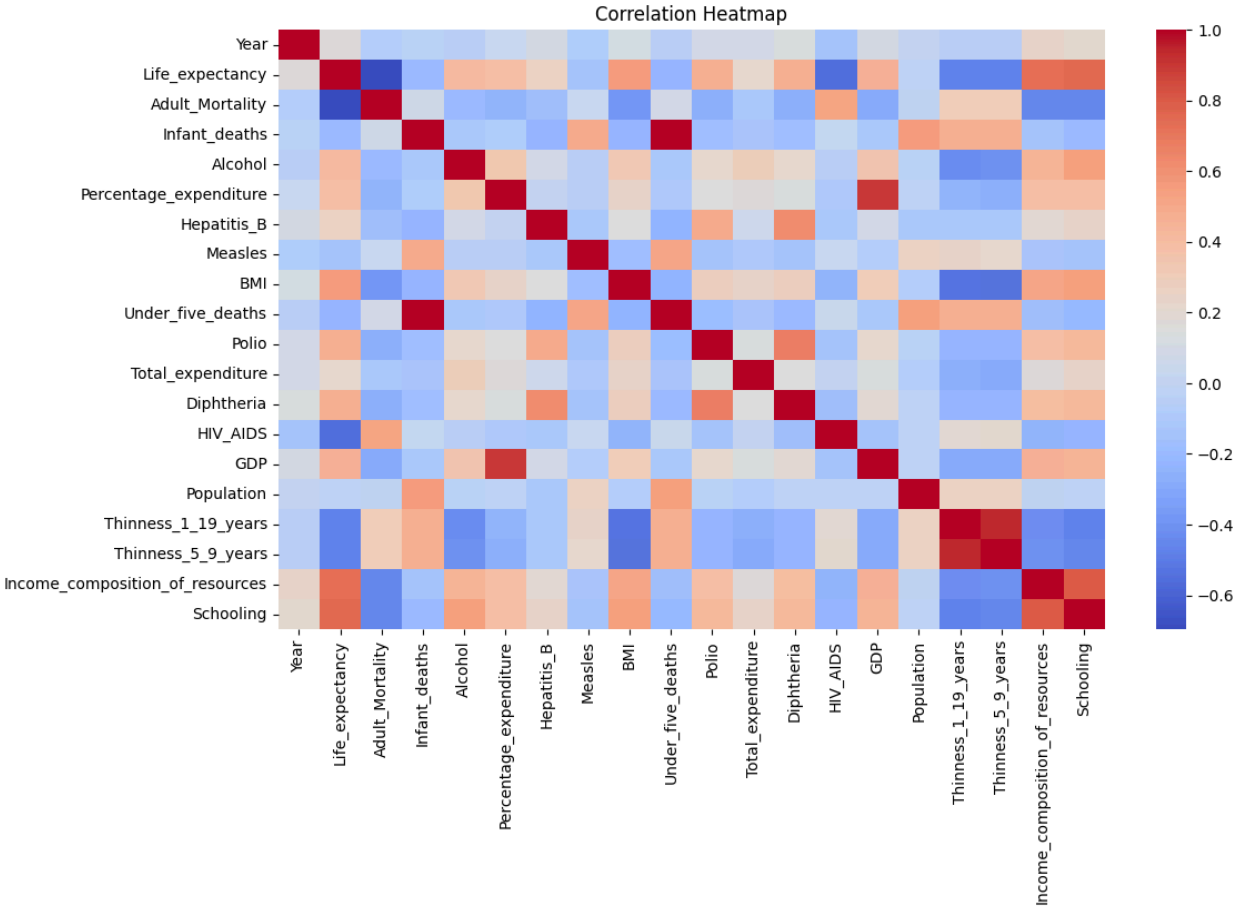
In [ ]:
```
# 7. Barplot
plt.figure(figsize=(7,5))
sns.barplot(x=df['Status'], y=df['Life_expectancy'])
plt.title("Life Expectancy by Status")
plt.show()
```



In [ ]:
```
# 12. MULTI-VARIATE VISUALIZATION
# 8. Pairplot
sns.pairplot(df[['Life_expectancy','GDP','Adult_Mortality','Alcohol']])
plt.show()
```



In [ ]:
```
# 9. Correlation Heatmap
plt.figure(figsize=(12,7))
sns.heatmap(df.corr(numeric_only=True), annot=False, cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.show()
```



In [ ]:
```
# 13. UNIQUE VALUES & VALUE COUNTS
df['Status'].unique()
```

Out[ ]:
```
array(['Developing', 'Developed'], dtype=object)
```

In [ ]:
```
df['Status'].value_counts()
```

Out[ ]:

Status	count
Developing	2426
Developed	512

dtype: int64

In [ ]:
```
df['Country'].nunique()
```

Out[ ]:
```
193
```

In [ ]:
```
# 14. HANDLE MISSING VALUES
cols = [
    'Life_expectancy',
```

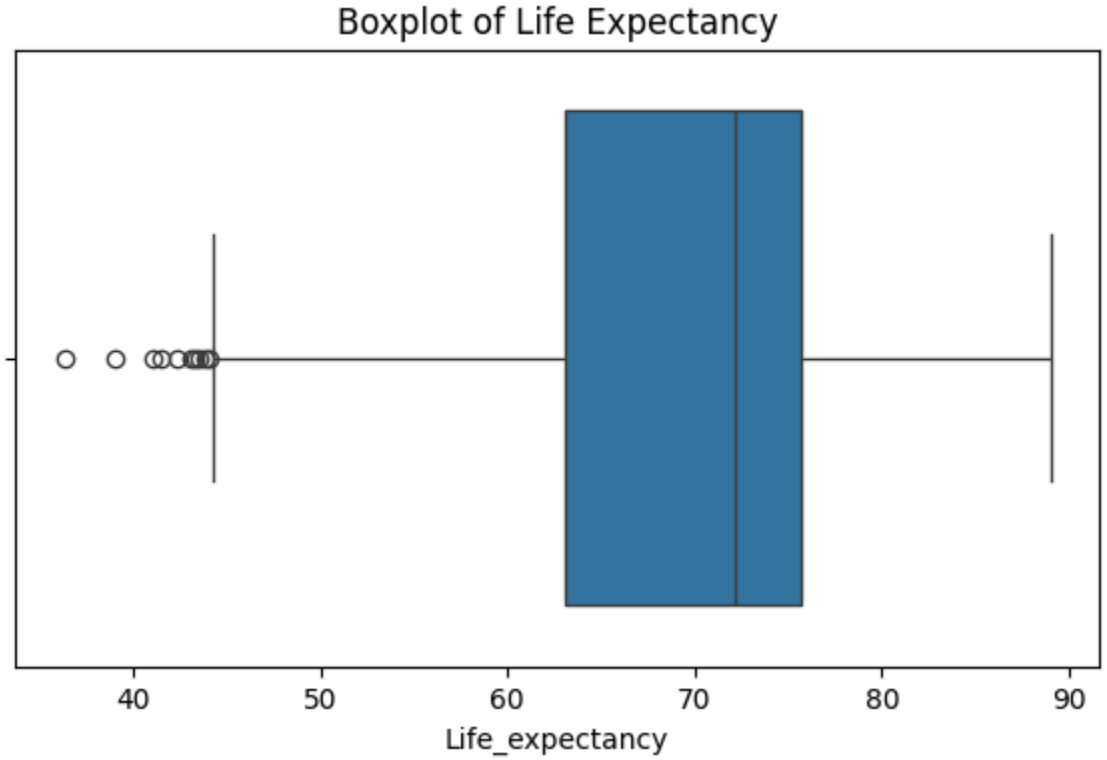
```
    'Adult_Mortality',
    'Alcohol',
    'Hepatitis_B',
    'BMI',
    'Polio',
    'Total_expenditure',
    'Diphtheria',
    'GDP',
    'Population',
    'Thinness_1_19_years',
    'Thinness_5_9_years',
    'Income_composition_of_resources',
    'Schooling'
]

# Fill missing numerical values using MEDIAN
df.loc[:,cols] = df[cols].fillna(df[cols].median())
```

In [ ]:
```
df.loc[:,'Country'] = df['Country'].fillna("Unknown")
df.loc[:,'Status'] = df['Status'].fillna("Unknown")
```

In [ ]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 2918 entries, 0 to 2937
Data columns (total 22 columns):
 #   Column                           Non-Null Count  Dtype  
---  ------                           --------------  -----  
 0   Country                          2918 non-null   int64  
 1   Year                             2918 non-null   int64  
 2   Status                           2918 non-null   int64  
 3   Life_expectancy                  2918 non-null   float64
 4   Adult_Mortality                  2918 non-null   float64
 5   Infant_deaths                    2918 non-null   int64  
 6   Alcohol                          2918 non-null   float64
 7   Percentage_expenditure           2918 non-null   float64
 8   Hepatitis_B                      2918 non-null   float64
 9   Measles                          2918 non-null   int64  
 10  BMI                              2918 non-null   float64
 11  Under_five_deaths                2918 non-null   int64  
 12  Polio                            2918 non-null   float64
 13  Total_expenditure                2918 non-null   float64
 14  Diphtheria                       2918 non-null   float64
 15  HIV_AIDS                         2918 non-null   float64
 16  GDP                              2918 non-null   float64
 17  Population                       2918 non-null   float64
 18  Thinness_1_19_years              2918 non-null   float64
 19  Thinness_5_9_years               2918 non-null   float64
 20  Income_composition_of_resources  2918 non-null   float64
 21  Schooling                        2918 non-null   float64
dtypes: float64(16), int64(6)
memory usage: 524.3 KB
```

In [ ]:
```
# 15. OUTLIER DETECTION
# Boxplot
plt.figure(figsize=(7,4))
sns.boxplot(x=df['Life_expectancy'])
plt.title("Boxplot of Life Expectancy")
plt.show()
```



In [ ]:
```
# Basic outlier detection using IQR for Life Expectancy
Q1 = df['Life_expectancy'].quantile(0.25)
Q3 = df['Life_expectancy'].quantile(0.75)
IQR = Q3 - Q1

lower_limit = Q1 - 1.5*IQR
upper_limit = Q3 + 1.5*IQR
```

In [ ]:
```
# Removing outliers
df = df[(df['Life_expectancy'] >= lower_limit) & (df['Life_expectancy'] <= upp
```

In [ ]:
```
# Boxplot (RECHECK)
plt.figure(figsize=(7,4))
sns.boxplot(x=df['Life_expectancy'])
plt.title("Boxplot of Life Expectancy")
plt.show()
```

In [ ]:
```
# RECHECK
df.head()
```

Out[ ]:

	Country	Year	Status	Life_expectancy	Adult_Mortality	Infant_deaths
0	Afghanistan	2015	Developing	65.0	263.0	62
1	Afghanistan	2014	Developing	59.9	271.0	64
2	Afghanistan	2013	Developing	59.9	268.0	66
3	Afghanistan	2012	Developing	59.5	272.0	69
4	Afghanistan	2011	Developing	59.2	275.0	71

In [ ]:
```
```

STEP 2- REGRESSION MODELS AND ENSEMBLE TECHNIQUES

In [ ]:
```
# 1. LABEL ENCODING
le = LabelEncoder()

df.loc[:, 'Status'] = le.fit_transform(df['Status'])
df.loc[:, 'Country'] = le.fit_transform(df['Country'])
```

In [ ]:
```
# 2. DEFINE FEATURES (X) & TARGET (Y)
# Defining target variable
Y = df['Life_expectancy']

# Defining feature variables directly using column names
X = df[['Country',
        'Year',
        'Status',
        'Adult_Mortality',
        'Infant_deaths',
        'Alcohol',
        'Percentage_expenditure',
        'Hepatitis_B',
        'Measles',
        'BMI',
        'Under_five_deaths',
        'Polio',
        'Total_expenditure',
        'Diphtheria',
        'HIV_AIDS',
        'GDP',
        'Population',
        'Thinness_1_19_years',
        'Thinness_5_9_years',
        'Income_composition_of_resources',
        'Schooling']]
```

In [ ]:
```
# 3. TRAIN-TEST SPLIT ( train= 80% and test=20%)
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.20, rand
```

In [ ]:
```
# 4. SCALING FOR MODELS THAT NEEDS IT
sc = StandardScaler()
X_train_s = sc.fit_transform(X_train)
X_test_s = sc.transform(X_test)
```

In [ ]:
```
# LINEAR REGRESSION
# Fits a straight line to predict values based on linear relationships.
lr = LinearRegression()
lr.fit(X_train, Y_train)
print("Linear Regression R2 Score:", r2_score(Y_test, lr.predict(X_test)))
```

```
Linear Regression R2 Score: 0.8204734657046557
```

In [ ]:
```
# Decision Tree
# Splits data into decision trees to make predictions.
dt = DecisionTreeRegressor()
dt.fit(X_train, Y_train)
print("Decision Tree R2 Score:", r2_score(Y_test, dt.predict(X_test)))
```

```
Decision Tree R2 Score: 0.9297426600211219
```

In [ ]:
```
# Random Forest
# Uses many decision trees and averages results for higher accuracy.
```

```
rf = RandomForestRegressor()
rf.fit(X_train, Y_train)
print("Random Forest R2 Score:", r2_score(Y_test, rf.predict(X_test)))
```

```
Random Forest R2 Score: 0.9591703802186614
```

In [ ]:
```
# KNN
# Predicts the value based on the average of nearest data points.
knn = KNeighborsRegressor()    # Nearest neighbor average
knn.fit(X_train, Y_train)
print("KNN R2 Score:", r2_score(Y_test, knn.predict(X_test_s)))
```

```
KNN R2 Score: 0.8906281322907659
```

In [ ]:
```
# SVR
# Fits the best possible curve within a margin for regression.
svr = SVR()
svr.fit(X_train_s, Y_train)
print("SVR R2 Score:", r2_score(Y_test, svr.predict(X_test_s)))
```

```
SVR R2 Score: 0.8462745403253613
```

In [ ]:
```
# Gradient Boosting
#Builds models step-by-step, improving errors of the previous model.
gbr = GradientBoostingRegressor()
gbr.fit(X_train, Y_train)
print("Gradient Boosting R2 Score:", r2_score(Y_test, gbr.predict(X_test)))
```

In [ ]:
```
# AdaBoost
# Learns from mistakes by giving more weight to difficult samples.
ada = AdaBoostRegressor()
ada.fit(X_train, Y_train)
print("AdaBoost R2 Score:", r2_score(Y_test, ada.predict(X_test)))
```

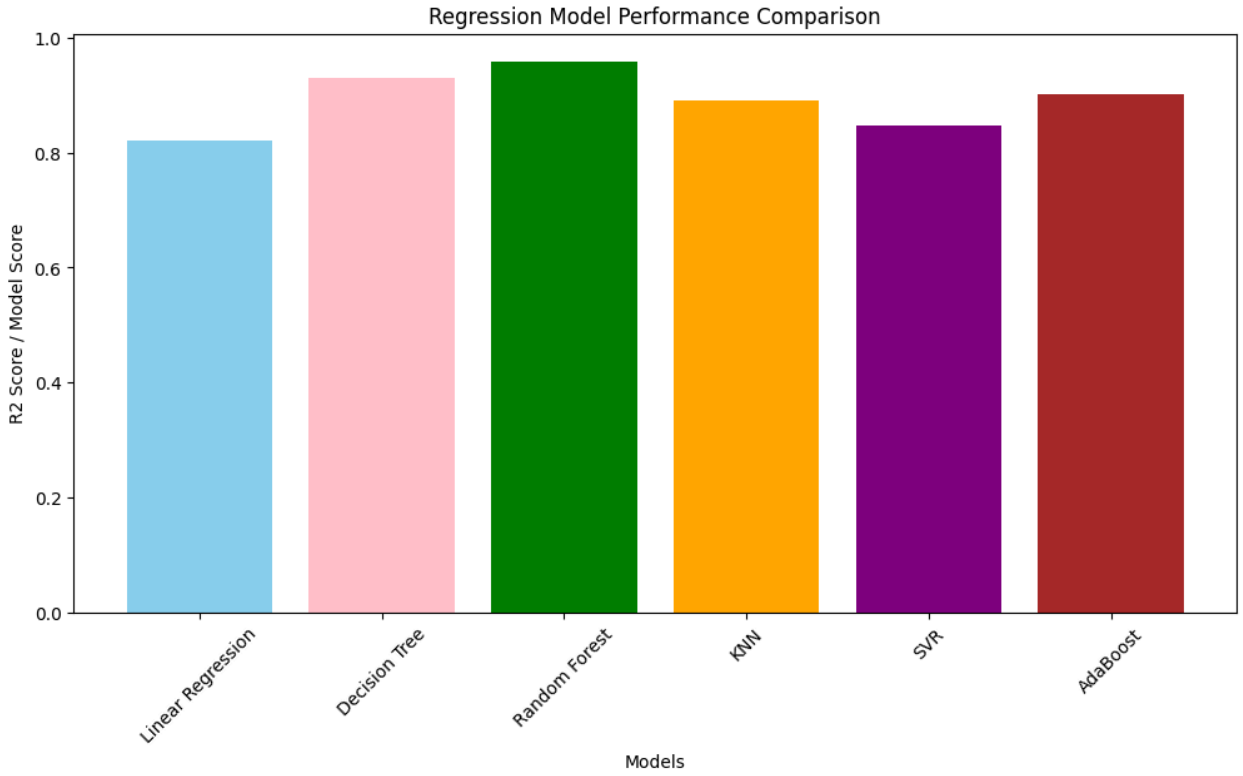
```
AdaBoost R2 Score: 0.9019592562683898
```

In [ ]:
```
# Calculate R2 scores for each trained model
lr_r2 = r2_score(Y_test, lr.predict(X_test))
dt_r2 = r2_score(Y_test, dt.predict(X_test))
rf_r2 = r2_score(Y_test, rf.predict(X_test))
knn_r2 = r2_score(Y_test, knn.predict(X_test_s))
svr_r2 = r2_score(Y_test, svr.predict(X_test_s))
ada_r2 = r2_score(Y_test, ada.predict(X_test))


model_results = {
    "Linear Regression": lr_r2,
    "Decision Tree Regressor": dt_r2,
    "Random Forest Regressor": rf_r2,
    "KNN Regressor": knn_r2,
    "SVR": svr_r2,
    "AdaBoost Regressor": ada_r2,

}
```

In [ ]:
```
model_results
```

Out[ ]:
```
{'Linear Regression': 0.8204734657046557,
 'Decision Tree Regressor': 0.9297426600211219,
 'Random Forest Regressor': 0.9591703802186614,
 'KNN Regressor': 0.8906281322907659,
 'SVR': 0.8462745403253613,
 'AdaBoost Regressor': 0.9019592562683898}
```

In [1]:
```
# STEP: BAR GRAPH FOR REGRESSION MODEL PERFORMANCE COMPARISON

models = [
    'Linear Regression',
    'Decision Tree',
    'Random Forest',
    'KNN',
    'SVR',
    'AdaBoost'
]

scores = [
    0.8204734657046557,   # Linear Regression
    0.9297426600211219,   # Decision Tree Regressor
    0.9591703802186614,   # Random Forest Regressor
    0.8906281322907659,   # KNN Regressor
    0.8462745403253613,   # SVR
    0.9019592562683898   # AdaBoost Regressor
]

import matplotlib.pyplot as plt

colors = ['skyblue', 'pink', 'green', 'orange', 'purple', 'brown']

plt.figure(figsize=(12,6))
plt.bar(models, scores, color=colors)
plt.xlabel("Models")
plt.ylabel("R2 Score / Model Score")
plt.title("Regression Model Performance Comparison")
plt.xticks(rotation=45)
plt.show()
```



INSIGHTS-

Higher R^2 = Better prediction accuracy

Lower R^2 = Model is not explaining the data well

- Random Forest (0.9591703802186614) = Best model
- They give the most correct predictions
- They explain the data more clearly

In [ ]: